

LAPORAN TUGAS BESAR

IF2211 - Strategi Algoritma

Implementasi Algoritma Greedy pada Robocode Tank Royale



Disusun oleh:

Cornelius Adhi Prasetya	Diki Hurrisyail	Agit Fadillah
124140052	124140088	124140157

Kelompok: [Ujang Kedu]

Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera
2026

DAFTAR ISI

BAB 1 PENDAHULUAN.....	4
1.1 Deskripsi Tugas.....	4
1.1.1 Isi Laporan.....	5
1.1.2 Keterangan Laporan.....	5
BAB 2 LANDASAN TEORI.....	6
2.1 Algoritma Greedy.....	6
2.1.2 Pseudocode Umum Algoritma Greedy.....	7
2.2 Robocode Tank Royale.....	7
2.2.1 Cara Kerja Bot.....	8
2.2.2 Implementasi Algoritma Greedy pada Bot.....	8
2.2.3 Cara Menjalankan Bot.....	8
BAB 3 APLIKASI STRATEGI GREEDY.....	10
3.1 Mapping Persoalan Robocode Tank Royale ke Algoritma Greedy.....	10
3.2 Eksplorasi Alternatif Solusi Greedy.....	10
3.2.1 Alternatif 1: [Adaptive Greedy Combat Strategy].....	10
3.2.2 Alternatif 2: [Distance-Based Greedy Combat Strategy].....	11
3.2.3 Alternatif 3: [Aggressive Pursuit Strategy].....	12
3.2.4 Alternatif 4: [Predictive Dynamic Combat Strategy].....	12
3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi.....	12
3.4 Strategi Greedy yang Dipilih.....	14
3.4.1 Alasan Pemilihan Strategi.....	14
BAB 4 IMPLEMENTASI DAN PENGUJIAN.....	16
4.1 Implementasi Keempat Alternatif Solusi.....	16
4.1.1 Pseudocode Alternatif 1: [Adaptive Greedy Combat Strategy].....	16
4.1.2 Pseudocode Alternatif 2: [Distance-Based Greedy Combat Strategy].....	17
4.1.3 Pseudocode Alternatif 3: [Aggressive Pursuit].....	18
4.1.4 Pseudocode Alternatif 4: [Predictive Dynamic Combat].....	19
4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur.....	21
4.2.1 Struktur Data.....	21
4.2.2 Fungsi Utama.....	22
4.2.3 Prosedur.....	23
4.3 Pengujian.....	23
4.3.1 Pengujian 1: [Bot Utama vs Bot Alternatif 2].....	23
4.3.2 Pengujian 2: [Bot Utama vs Bot Alternatif 3].....	24
4.3.3 Pengujian 3: [Bot Utama vs Bot Alternatif 4].....	24
4.4 Analisis Hasil Pengujian.....	25
BAB 5 KESIMPULAN DAN SARAN.....	28
5.1 Kesimpulan.....	28

5.2 Saran.....	28
LAMPIRAN.....	30
Lampiran A: Tautan Repository GitHub.....	30
Lampiran B: Tautan Video (jika ada).....	30
DAFTAR PUSTAKA.....	31

BAB 1 PENDAHULUAN

1.1 Deskripsi Tugas

Berikut adalah spesifikasi tugas yang diberikan:

Spesifikasi Wajib

- Buatlah 4 bot (1 utama dan 3 alternatif, **tetap dikumpulkan 1 bot yang terbaik saja**) dalam bahasa C# (.net) yang mengimplementasikan *algoritma Greedy* pada bot permainan Robocode Tank Royale dengan tujuan memenangkan permainan.
- Tugas dikerjakan berkelompok dengan anggota **minimal 2 orang** dan **maksimal 3 orang**.
- Strategi *greedy* yang diimplementasikan setiap kelompok harus dikaitkan dengan fungsi objektif dari permainan ini, yaitu memperoleh skor setinggi mungkin pada akhir pertempuran. Hal ini dapat dilakukan dengan mengoptimalkan komponen skor yang telah dijelaskan diatas.
- Strategi *greedy* yang diimplementasikan **harus berbeda** untuk setiap bot yang diimplementasikan dan setiap strategi *greedy* harus menggunakan **heuristic** yang berbeda.
- Bot yang dibuat **TIDAK BOLEH** sama dengan **SAMPEL** yang diberikan sebagai **CONTOH**. Baik dari starter pack maupun dari repository engine asli.
- Buatlah strategi *greedy* terbaik, karena setiap **bot utama** dari masing-masing kelompok akan diadu dalam kompetisi Tubes.
- Strategi *greedy* yang kelompok anda buat harus **dijelaskan dan ditulis secara eksplisit** pada laporan, karena akan diperiksa saat demo apakah strategi yang dituliskan sesuai dengan yang diimplementasikan.
- Setiap kelompok dapat menggunakan kreativitas yang bermacam macam dalam menyusun strategi *greedy* untuk memenangkan permainan. Implementasi pemain **harus dapat dijalankan pada game engine** yang telah disebutkan diatas serta dapat dikompetisikan dengan bot dari kelompok lain.

- Program harus mengandung komentar yang jelas, dan untuk setiap strategi *greedy* yang disebutkan, harus dilengkapi dengan **kode sumber yang dibuat**. Artinya semua strategi harus diimplementasikan
- Mahasiswa disarankan membaca [dokumentasi](#) dari *game engine*. Perlu diperhatikan bahwa game engine yang digunakan untuk tubes ini **SUDAH DIMODIFIKASI**. Untuk Perubahan dapat dilihat pada bagian starter pack.

1.1.1 Isi Laporan

- Cover: Cover laporan ada foto anggota kelompok (foto bertiga).
- Bab 1: Deskripsi tugas (dapat menyalin spesifikasi tugas ini).
- Bab 2: Landasan Teori — Dasar teori (algoritma greedy) secara umum; bagaimana cara kerja program secara umum (bagaimana bot melakukan aksinya, bagaimana mengimplementasikan algoritma greedy ke dalam bot, bagaimana menjalankan bot, dll).
- Bab 3: Aplikasi strategi greedy — Proses mapping persoalan Robocode Tank Royale menjadi elemen-elemen algoritma Greedy (himpunan kandidat, himpunan solusi, fungsi solusi, fungsi seleksi, fungsi kelayakan, fungsi objektif).
- Bab 3 (lanjutan): Eksplorasi 4 alternatif solusi greedy yang mungkin dipilih dalam persoalan Robocode Tank Royale (setiap alternatif harus memiliki heuristic yang berbeda). Analisis efisiensi dan efektivitas dari kumpulan alternatif solusi greedy. Strategi greedy yang dipilih beserta alasan dan pertimbangan.
- Bab 4: Implementasi dan pengujian — Implementasi 4 alternatif solusi (pseudocode detail). Penjelasan struktur data, fungsi, dan prosedur. Pengujian minimal 3 kali pertandingan 1 vs 1. Analisis hasil pengujian.
- Bab 5: Kesimpulan dan saran.
- Lampiran: Tautan repository GitHub dan video (jika membuat).
- Daftar Pustaka.

1.1.2 Keterangan Laporan

1. Laporan ditulis dalam bahasa Indonesia yang baik dan benar.
2. Laporan mengikuti format pada section "Isi laporan" dengan baik dan benar.
3. Identitas per halaman harus jelas (misalnya: halaman, kode kuliah).

BAB 2 LANDASAN TEORI

2.1 Algoritma Greedy

Greedy merupakan salah satu strategi penyelesaian masalah optimasi yang menyelesaikan masalah langkah demi langkah (step-by-step). Prinsip utama algoritma ini adalah mengambil pilihan yang terbaik pada setiap langkah lokal (local optimum) tanpa mempertimbangkan konsekuensi jangka panjang, dengan harapan bahwa rangkaian keputusan tersebut akan mengarah pada solusi terbaik global (global optimum). Menurut teori strategi algoritma, pendekatan Greedy disusun oleh lima elemen atau komponen utama berikut: Himpunan Kandidat (\$C\$): Berisi elemen-elemen yang tersedia untuk membentuk solusi. Pada Robocode, kandidat ini berupa seluruh koordinat posisi di arena, daftar objek musuh yang terdeteksi, atau opsi sudut rotasi meriam (gun). Himpunan Solusi (\$S\$): Kandidat yang telah dipilih pada setiap tahapan langkah. Contohnya adalah instruksi pergerakan operasional seperti `setAhead()`, `setTurnRight()`, atau `setFire()`. Fungsi Seleksi (Selection Function): Fungsi heuristik yang digunakan untuk memilih kandidat terbaik berdasarkan kriteria optimasi tertentu pada satu waktu spesifik. Fungsi Kelayakan (Feasible Function): Pengujian untuk memastikan bahwa kandidat yang dipilih tidak melanggar batasan sistem (constraints). Dalam konteks Robocode, kelayakan berarti robot tidak menabrak dinding arena atau tidak mengeksekusi tembakan saat energi berada di titik kritis ($E \leq 0$). Fungsi Objektif: Fungsi yang memaksimalkan atau meminimalkan nilai solusi akhir, seperti memaksimalkan total skor pertempuran (bullet damage, survival score) atau meminimalkan kerusakan yang diterima robot sendiri. Algoritma Greedy terdiri dari beberapa elemen utama, yaitu:

- Himpunan Kandidat (\$C\$): Sekumpulan elemen yang dapat dipilih untuk membentuk solusi.
- Himpunan Solusi (\$S\$): Subset dari himpunan kandidat yang telah dipilih untuk membentuk solusi.
- Fungsi Seleksi: Fungsi yang memilih kandidat terbaik dari himpunan kandidat yang belum dipilih.
- Fungsi Kelayakan: Fungsi yang memeriksa apakah suatu kandidat dapat ditambahkan ke solusi tanpa melanggar constraint.
- Fungsi Objektif: Fungsi yang mengukur kualitas solusi (nilai yang ingin dimaksimalkan atau diminimalkan).

- Fungsi Solusi: Fungsi yang memeriksa apakah himpunan solusi saat ini sudah merupakan solusi lengkap.

2.1.2 Pseudocode Umum Algoritma Greedy

```

FUNCTION Greedy(C: Himpunan Kandidat):
    S <- {} // Himpunan solusi kosong
    WHILE NOT Solusi(S) AND C != {} DO
        x <- Seleksi(C) // Pilih kandidat terbaik
        C <- C - {x}      // Hapus x dari kandidat
        IF Layak(S U {x}) THEN
            S <- S U {x}  // Tambahkan x ke solusi
        END IF
    END WHILE
    IF Solusi(S) THEN
        RETURN S          // Solusi ditemukan
    ELSE
        RETURN 'Tidak ada solusi'
    END IF
END FUNCTION

```

2.2 Robocode Tank Royale

Robocode Tank Royale adalah platform pemrograman game di mana pemain membuat bot tank yang bertarung satu sama lain dalam sebuah arena persegi. Bot diprogram menggunakan berbagai bahasa pemrograman (Java, .NET/C#, Python, dll.) dan berkompetisi secara otomatis tanpa interaksi manusia selama pertandingan berlangsung.

Arena pertempuran memiliki batas dinding yang tidak boleh dilampaui. Setiap bot memiliki komponen utama: badan (body) yang dapat bergerak, menara meriam (gun turret) yang dapat diputar secara independen, dan radar yang mendeteksi posisi musuh. Energi bot berkurang saat tertembak, menabrak dinding, atau menabrak bot lain. Bot dengan energi 0 dinyatakan kalah; bot terakhir yang bertahan menang.

Komponen skor dalam Robocode Tank Royale meliputi:

- Bullet Damage: Skor diperoleh dari damage yang diberikan kepada musuh melalui tembakan peluru.
- Bullet Kill Bonus: Bonus skor saat berhasil menghancurkan bot musuh dengan peluru.

- Ram Damage: Skor dari kerusakan akibat menabrak bot musuh secara langsung.
- Ram Kill Bonus: Bonus skor saat menghancurkan bot musuh melalui tabrakan.
- Survival Score: Skor bertahan yang diberikan setiap saat bot masih aktif di arena.
- Last Survivor Bonus: Bonus tambahan untuk bot terakhir yang bertahan.

2.2.1 Cara Kerja Bot

Bot dalam Robocode Tank Royale beroperasi dengan merespons berbagai event yang terjadi dalam permainan. Event-event tersebut antara lain:

- onScannedBot: Dipanggil ketika radar bot mendeteksi bot lawan.
- onHitBot: Dipanggil ketika bot menabrak bot lain.
- onHitWall: Dipanggil ketika bot menabrak dinding arena.
- onBulletHit: Dipanggil ketika peluru bot mengenai bot lawan.
- onTick: Dipanggil setiap tick (satuan waktu) dalam permainan.

2.2.2 Implementasi Algoritma Greedy pada Bot

Pada setiap tick (event onTick), bot mengevaluasi state permainan saat ini dan membuat keputusan greedy berdasarkan informasi terkini. Proses pengambilan keputusan greedy pada bot Robocode Tank Royale dilakukan sebagai berikut:

- Kumpulkan semua informasi state: posisi bot sendiri, daftar musuh yang terdeteksi (posisi, energi, heading), dan kondisi arena.
- Dari himpunan kandidat aksi (bergerak ke arah tertentu, memutar meriam, menembak dengan firepower tertentu), pilih aksi yang memberikan nilai terbaik menurut fungsi heuristik.
- Periksa kelayakan aksi: pastikan tidak menabrak dinding, energi mencukupi untuk menembak, dan tidak berada dalam situasi berbahaya.
- Eksekusi aksi terpilih melalui API bot (setAhead, setTurnRight, setGunTurnRight, setFire, dll.).
- Setelah semua aksi didaftarkan, panggil execute() untuk mengirimkan aksi ke engine.

2.2.3 Cara Menjalankan Bot

Berikut adalah langkah-langkah untuk menjalankan bot Robocode Tank Royale pada platform yang telah dimodifikasi:

1. Pastikan .NET SDK (versi 6.0 atau lebih baru) telah terinstal pada sistem.

2. Clone atau unduh starter pack dari repository yang diberikan oleh dosen.
3. Buka terminal/command prompt dan navigasi ke direktori bot yang diinginkan.
4. Jalankan perintah: `dotnet build` untuk mengompilasi bot.
5. Unduh dan ekstrak Robocode Tank Royale GUI (`robocode-tankroyale-gui-*.jar`).
Jalankan dengan: `java -jar robocode-tankroyale-gui-*.jar`
6. Pada GUI, pilih menu 'Config' > 'Bot Root Directories' dan tambahkan path ke direktori hasil build bot.
7. Buat battle baru dengan memilih 'Battle' > 'Start Battle', pilih bot yang akan bertarung, dan klik 'Start Battle'.
8. Untuk menjalankan dari command line (tanpa GUI), gunakan perintah: `dotnet run -- --server-url=ws://localhost:7654 --server-secret=[secret]`

BAB 3 APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan Robocode Tank Royale ke Algoritma Greedy

Berikut adalah pemetaan elemen-elemen persoalan Robocode Tank Royale ke dalam komponen algoritma Greedy:

Elemen Greedy	Pemetaan dalam Robocode Tank Royale
Himpunan Kandidat (C)	Seluruh aksi yang mungkin dilakukan bot pada setiap tick: (1) arah gerakan (maju/mundur/berhenti dengan berbagai jarak), (2) sudut rotasi badan, (3) sudut rotasi meriam, (4) sudut rotasi radar, (5) firepower tembakan (0.1 s/d 3.0), (6) keputusan untuk tidak menembak.
Himpunan Solusi (S)	Aksi-aksi yang telah dipilih dan akan dieksekusi dalam satu tick: kombinasi dari <code>setAhead(dist)</code> , <code>setTurnRight(deg)</code> , <code>setGunTurnRight(deg)</code> , <code>setRadarTurnRight(deg)</code> , dan <code>setFire(power)</code> .
Fungsi Seleksi	Fungsi heuristik yang memilih aksi terbaik berdasarkan kriteria tertentu, misalnya: pilih musuh dengan energi terendah sebagai target, atau pilih posisi yang memaksimalkan jarak dari semua musuh aktif.
Fungsi Kelayakan	Batasan fisik dan aturan permainan: (1) posisi hasil gerakan tidak melebihi batas arena, (2) energi bot > 0.1 untuk dapat menembak, (3) $\text{firepower} \leq \text{energi bot} / 2$ agar tidak habis energi, (4) meriam tidak sedang dalam periode cool-down.
Fungsi Objektif	Memaksimalkan total skor pertempuran: $\text{bullet_damage} + \text{bullet_kill_bonus} + \text{survival_score} + \text{last_survivor_bonus}$, sambil meminimalkan kerusakan yang diterima (damage_taken).
Fungsi Solusi	Kondisi yang menandakan solusi lengkap: hanya satu bot yang tersisa di arena ($\text{NumberOfParticipants} == 1$), yaitu kondisi kemenangan bot kita.

3.2 Eksplorasi Alternatif Solusi Greedy

Berikut adalah empat alternatif solusi greedy yang dipertimbangkan, masing-masing dengan heuristic yang berbeda:

3.2.1 Alternatif 1: [Adaptive Greedy Combat Strategy]

Strategi ini menggunakan pendekatan greedy adaptif yang mengambil keputusan terbaik secara lokal pada setiap tick berdasarkan kondisi arena. Bot secara simultan melakukan scanning, movement, aiming, dan firing. Prioritas keputusan berubah mengikuti kondisi saat itu.

Bot memanfaatkan radar lock untuk terus memantau musuh, predictive aiming untuk menembak posisi masa depan lawan, movement orbit untuk menghindari tembakan, serta anti-wall behavior agar tidak terjebak di pinggir arena.

Heuristic:

Nilai prioritas tindakan dihitung berdasarkan:

$$H(x) = \text{Attack Value} + \text{Survival Value} + \text{Position Value}$$

dengan:

- Attack Value → peluang mengenai musuh
- Survival Value → kemungkinan menghindari peluru
- Position Value → keamanan posisi dalam arena

Pada setiap tick dipilih aksi dengan nilai heuristik tertinggi.

3.2.2 Alternatif 2: [Distance-Based Greedy Combat Strategy]

Strategi ini menggunakan pendekatan greedy berbasis jarak musuh (distance-based heuristic). Pada setiap tick, bot akan menentukan keputusan terbaik berdasarkan kondisi lokal berupa jarak target terhadap bot.

Bot menerapkan pergerakan zigzag terus-menerus untuk meningkatkan peluang bertahan hidup, kemudian menggunakan radar sweep untuk menemukan musuh secepat mungkin. Setelah musuh ditemukan, bot memilih firepower secara greedy berdasarkan jarak lawan. Semakin dekat musuh, semakin besar firepower yang dipilih karena peluang mengenai target lebih tinggi. Sebaliknya, ketika musuh jauh, bot menggunakan firepower kecil agar peluru bergerak lebih cepat dan energi lebih hemat.

Heuristic:

Heuristic yang digunakan:

$$H(x) = \text{Damage Potential} + \text{Survival Probability}$$

dengan:

- Damage Potential → estimasi peluang menghasilkan damage berdasarkan jarak
- Survival Probability → peluang menghindari serangan dengan gerakan zigzag

Pemilihan aksi selalu mengambil nilai heuristic terbesar pada setiap tick.

3.2.3 Alternatif 3: [Aggressive Pursuit Strategy]

Strategi ini menggunakan pendekatan greedy agresif dengan memprioritaskan pengejaran musuh secepat mungkin. Ketika radar mendeteksi target, bot langsung mengarahkan badan menuju posisi musuh, mendekat secara agresif, lalu menggunakan firepower maksimum.

Bot juga menerapkan mekanisme anti-wall escape untuk menghindari kondisi terjebak di tepi arena. Apabila bot terkena peluru, dilakukan manuver acak sederhana agar gerakan tidak terlalu mudah diprediksi.

Heuristic:

$$H(x) = \text{Chase Priority} + \text{Attack Opportunity}$$

dengan:

- Chase Priority → prioritas mendekati target secepat mungkin
- Attack Opportunity → peluang menembak berdasarkan arah gun dan posisi musuh

Pada setiap tick dipilih aksi yang paling cepat menghasilkan kontak dengan musuh.

3.2.4 Alternatif 4: [Predictive Dynamic Combat Strategy]

Strategi ini menggunakan kombinasi greedy dan prediksi pergerakan musuh. Pada setiap tick, bot memilih aksi lokal terbaik berdasarkan posisi musuh, arah gerakan lawan, jarak tempur ideal, dan keamanan posisi arena.

Bot menerapkan orbit movement, distance control, wall smoothing, predictive targeting, serta anti-targeting dodge agar lebih sulit diprediksi lawan.

Heuristic:

$$H(x) = \text{Prediction Accuracy} + \text{Position Safety} + \text{Attack Potential}$$

dengan:

- Prediction Accuracy → akurasi prediksi posisi musuh
- Position Safety → tingkat keamanan posisi terhadap dinding dan peluru
- Attack Potential → peluang menghasilkan damage

Bot memilih aksi dengan nilai heuristik tertinggi pada setiap tick.

3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi

Berikut adalah perbandingan analisis dari keempat alternatif solusi greedy:

Alternatif	Heuristic	Efisiensi	Efektivitas	Keterangan
------------	-----------	-----------	-------------	------------

Alternatif 1 Adaptive Greedy Combat	Attack + Survival + Positioning	$O(1)$	Tinggi	Strategi adaptif lengkap
Alternatif 2 Distance-Based Combat	Damage Potential + Survival Probability	$O(1)$	Sedang - Tinggi	Fokus pada jarak target
Alternatif 3 Aggressive Pursuit	Chase Priority + Attack Opportunity	$O(1)$	Sedang	Fokus mengejar musuh
Alternatif 4 Predictive Dynamic Combat	Prediction + Safety + Attack	$O(1)$	Sangat Tinggi	Prediksi + movement adaptif

- Strategi memiliki kompleksitas $O(1)$ karena seluruh keputusan hanya memanfaatkan data musuh saat ini tanpa pencarian global atau penyimpanan riwayat panjang. Setiap tick bot hanya melakukan operasi aritmatika sederhana seperti menghitung jarak, arah, dan prediksi posisi musuh. Dari sisi efektivitas, strategi cukup baik karena menggabungkan beberapa keputusan greedy secara bersamaan, yaitu movement, aiming, radar, dan firing. Namun performanya dapat menurun ketika melawan bot dengan pergerakan acak ekstrem karena predictive aiming menjadi kurang akurat.
- Alternatif kedua memiliki kompleksitas $O(1)$ karena keputusan hanya menggunakan parameter lokal berupa posisi musuh dan jarak. Tidak terdapat pencarian state atau proses optimasi global. Perhitungan yang digunakan hanya operasi matematika sederhana pada setiap tick. Dari sisi efektivitas, strategi ini cukup stabil pada pertandingan satu lawan satu karena mekanisme zigzag membuat bot lebih sulit diprediksi. Namun strategi ini memiliki keterbatasan karena aiming masih menggunakan posisi musuh saat ini dan belum menerapkan prediksi gerakan musuh.
- Alternatif ketiga memiliki kompleksitas $O(1)$ karena keputusan hanya menggunakan posisi musuh terakhir, arah bot, dan kondisi dinding arena. Tidak terdapat pencarian state atau simulasi prediksi yang kompleks. Strategi ini efektif untuk pertandingan satu lawan satu karena memberikan tekanan terus-menerus kepada lawan, namun cenderung berisiko karena pergerakan agresif dapat membuat bot lebih mudah terkena serangan.
- Alternatif keempat memiliki kompleksitas $O(1)$ karena setiap tick hanya menggunakan data posisi, arah, kecepatan musuh, serta operasi matematika sederhana. Tidak terdapat pencarian lintasan atau eksplorasi state. Walaupun memiliki proses prediksi posisi

masa depan, perhitungan tetap dilakukan dalam jumlah operasi konstan. Dari sisi efektivitas, strategi ini diperkirakan paling baik karena menggabungkan predictive targeting, orbit movement, pengaturan jarak dinamis, dan anti-wall movement. Strategi ini menghasilkan keseimbangan antara agresivitas dan survival.

3.4 Strategi Greedy yang Dipilih

Dari keempat alternatif solusi greedy yang telah dieksplorasi, strategi yang dipilih untuk diimplementasikan adalah:

Nama Strategi: Adaptive Greedy Combat (Alternatif ke-1)

Strategi Adaptive Greedy Combat dipilih karena mampu menggabungkan beberapa mekanisme pengambilan keputusan greedy secara bersamaan, yaitu movement control, predictive targeting, distance control, energy management, bullet dodge, dan anti-wall behavior. Strategi ini tidak hanya berfokus pada serangan, tetapi juga mempertimbangkan aspek bertahan hidup dan positioning di arena.

3.4.1 Alasan Pemilihan Strategi

Strategi Adaptive Greedy Combat dipilih karena memiliki pendekatan yang lebih seimbang dibandingkan alternatif lainnya. Pada setiap tick, bot melakukan evaluasi kondisi lokal yang sedang terjadi dan memilih aksi yang dianggap paling menguntungkan pada saat itu. Pendekatan ini sesuai dengan karakteristik Robocode yang membutuhkan pengambilan keputusan cepat dalam lingkungan real-time.

Dibandingkan dengan Alternatif 2 yang lebih berfokus pada pemilihan firepower berdasarkan jarak, strategi ini memiliki mekanisme yang lebih lengkap karena tidak hanya mempertimbangkan serangan tetapi juga movement dan pengelolaan energi. Alternatif 3 memiliki keunggulan pada agresivitas pengejaran target, namun cenderung terlalu berisiko karena bot sering bergerak langsung menuju musuh sehingga lebih mudah terkena serangan. Sementara Alternatif 4 memang memiliki kemampuan prediksi dan movement yang sangat baik, tetapi implementasinya lebih kompleks dan lebih sulit dikontrol karena banyak parameter yang saling memengaruhi.

Strategi Adaptive Greedy Combat dipilih karena menawarkan keseimbangan antara efektivitas serangan, kemampuan bertahan hidup, dan kompleksitas implementasi. Strategi ini tetap mampu melakukan prediksi posisi musuh, menghindari peluru, mengatur

jarak tempur, serta mencegah bot terjebak di area dinding tanpa menambah kompleksitas algoritma secara berlebihan.

Kelebihan strategi Adaptive Greedy Combat:

- Memiliki mekanisme predictive targeting untuk meningkatkan akurasi tembakan.
- Mampu melakukan dodge saat mendeteksi kemungkinan tembakan musuh.
- Menerapkan distance control sehingga bot dapat menjaga posisi tempur yang lebih aman.
- Memiliki anti-wall movement agar tidak mudah terjebak di tepi arena.
- Menggunakan energy management sehingga penggunaan peluru lebih efisien.

Kekurangan strategi Adaptive Greedy Combat:

- Prediksi posisi musuh masih menggunakan pendekatan linear sederhana.
- Mekanisme deteksi peluru berdasarkan penurunan energi musuh tidak selalu akurat.
- Performa dapat menurun ketika menghadapi musuh dengan pola gerakan sangat acak.
- Memiliki lebih banyak parameter dibanding strategi sederhana sehingga memerlukan proses tuning yang lebih besar.

Berdasarkan pertimbangan tersebut, strategi Adaptive Greedy Combat dipilih karena memberikan keseimbangan terbaik antara agresivitas, survival, dan efisiensi dalam proses pengambilan keputusan greedy.

BAB 4 IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Keempat Alternatif Solusi

4.1.1 Pseudocode Alternatif 1: [Adaptive Greedy Combat Strategy]

```
// Pseudocode Alternatif 1
FUNCTION AdaptiveGreedyCombat()

    WHILE bot hidup

        moveTick++

        IF dekatDinding THEN
            arahKeTengah()
            maju(160)
            CONTINUE
        ENDIF

        IF fireCooldown>0 THEN
            fireCooldown--
        ENDIF

        putarRadar(45)

        IF enemyDetected THEN

            IF bulletDetected THEN
                moveDirection*=-1
                belok(70*moveDirection+random(-15,15))
                maju(60)
                bulletDetected=false
            ELSE
                IF jarakMusuh>400 THEN
                    maju(80)

                ELSE IF jarakMusuh>180 THEN
                    maju(90)

                ELSE
                    belok(45*moveDirection)
                    maju(60)
                ENDIF

                IF moveTick mod 30=0 THEN
                    moveDirection*=-1
                ENDIF
            ENDIF
        ENDIF
```



```

ELSE
    belok(15)
    maju(80)
ENDIF

IF enemyDetected THEN

    IF jarakMusuh<120 THEN
        firePower←3

    ELSE IF jarakMusuh<300 THEN
        firePower←2

    ELSE
        firePower←1
    ENDIF

    prediksiPosisiMusuh()

    putarGun()

    IF aimPresisi AND gunReady AND cooldown=0 THEN
        tembak(firePower)
        setCooldown()
    ENDIF

ENDIF

eksekusi()

ENDWHILE

END FUNCTION

```

4.1.2 Pseudocode Alternatif 2: [Distance-Based Greedy Combat Strategy]

```

// Pseudocode Alternatif 2
FUNCTION DistanceGreedyCombat()

    WHILE bot hidup

        moveCounter++

        IF moveCounter mod 20=0 THEN
            moveDirection*=-1
            belok(random(30,60))
        ENDIF

        IF moveDirection=1 THEN
            maju(80)
        ELSE

```

```
    mundur(80)
ENDIF

putarRadar(45)

IF enemyDetected AND energy>20 THEN

    IF jarak<200 THEN
        firePower←3

    ELSE IF jarak<400 THEN
        firePower←2

    ELSE
        firePower←1
    ENDIF

    putarGunKeMusuh()

    IF aimPresisi AND gunReady THEN
        tembak(firePower)
    ENDIF

ENDIF

eksekusi()

ENDWHILE

END FUNCTION
```

4.1.3 Pseudocode Alternatif 3: [Aggressive Pursuit]

```
// Pseudocode Alternatif 3
FUNCTION AggressivePursuit()

    WHILE bot hidup

        putarRadar(360)

        IF escapeWall THEN

            mundur(120)

            arahKeTengah()

            escapeTick--

            IF escapeTick<=0 THEN
                escapeWall←false
            ENDIF

        ENDIF

    ENDWHILE

END FUNCTION
```

```

    eksekusi()

    CONTINUE

ENDIF

IF enemyDetected THEN

    arahMusuh()

    maju(200)

    putarGunKeMusuh()

    IF aimPresisi AND gunReady THEN
        tembak(3)
    ENDIF

ELSE

    belok(25)

    maju(100)

ENDIF

eksekusi()

ENDWHILE

END FUNCTION

```

4.1.4 Pseudocode Alternatif 4: [Predictive Dynamic Combat]

```

// Pseudocode Alternatif 4
FUNCTION PredictiveDynamicCombat()

    WHILE bot hidup

        IF enemyDetected=false THEN
            putarRadar(360)
        ENDIF

        ubahArahAcak()

        IF enemyDetected THEN

            orbitMusuh()

            IF jarakMusuh<180 THEN
                mundur(120)
            ENDIF
        ENDIF
    ENDWHILE
ENDFUNCTION

```

```
ELSE IF jarakMusuh>350 THEN
    maju(120)

ELSE
    maju(80×moveDirection)
ENDIF

ELSE

    maju(100)
    belok(20)

ENDIF

hindariDinding()

IF enemyDetected THEN

    IF jarakMusuh<150 THEN
        firePower←3

    ELSE IF jarakMusuh<300 THEN
        firePower←2

    ELSE
        firePower←1.2
    ENDIF

    IF energy<20 THEN
        firePower←1
    ENDIF

    prediksiPosisiMusuh()

    putarGun()

    IF aimPresisi AND gunReady THEN
        tembak(firePower)
    ENDIF

ENDIF

eksekusi()

ENDWHILE

END FUNCTION
```

4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur

Bagian ini menjelaskan struktur data, fungsi, dan prosedur yang digunakan pada bot dengan solusi greedy yang dipilih (Alternatif [1] UjangKeduDorDor).

4.2.1 Struktur Data

Implementasi strategi menggunakan beberapa variabel untuk menyimpan informasi kondisi arena dan status permainan.

- **Data Musuh**

Digunakan untuk menyimpan informasi target hasil radar scan.

Variabel	Fungsi
ex	koordinat X musuh
ey	koordinat Y musuh
edist	jarak musuh
edir	arah gerakan musuh
espd	kecepatan musuh

Data ini digunakan pada proses movement, predictive aiming, dan distance control.

- **Status Deteksi Musuh**

Digunakan untuk mengetahui kondisi musuh dan ancaman.

Variabel	Fungsi
enemyDetected	status apakah musuh terdeteksi
bulletDetected	indikasi musuh menembak
enemyEnergy	energi musuh saat ini
lastEnemyEnergy	energi musuh sebelumnya

Penurunan energi musuh digunakan untuk memperkirakan kemungkinan peluru ditembakkan.

- **Movement Control**

Digunakan untuk mengatur pola gerakan bot.

Variabel	Fungsi
moveDir	arah pergerakan
moveTick	penghitung perubahan gerakan

Variabel ini digunakan pada orbit movement, dodge movement, dan random movement.

- **Fire Control**

Digunakan untuk mengatur proses penembakan.

Variabel	Fungsi
fireCooldown	jeda antar tembakan
lastFirePower	menyimpan firepower terakhir

Sistem cooldown digunakan untuk mengurangi penembakan berlebihan dan menjaga akurasi.

- **Reward System**

Digunakan sebagai indikator reward virtual.

Variabel	Fungsi
bonusEnergy	bonus agresivitas

Reward akan meningkat ketika peluru berhasil mengenai musuh.

4.2.2 Fungsi Utama

Implementasi Adaptive Greedy Combat dibagi menjadi beberapa komponen utama:

- **Radar Control**

Radar dipisahkan dari body dan gun agar proses scanning tetap berjalan meskipun bot sedang bergerak atau menembak. Sistem radar juga menggunakan radar lock agar target tetap terpantau.

- **Movement Control**

Bot menggunakan orbit movement dan zigzag movement untuk meningkatkan peluang menghindari peluru. Selain itu terdapat anti-wall movement untuk menghindari posisi berbahaya di tepi arena.

- **Targeting System**

Sistem targeting menggunakan predictive aiming dengan memperkirakan posisi masa depan musuh berdasarkan arah dan kecepatan gerakan.

- **Fire Control**

Firepower dipilih menggunakan pendekatan greedy berdasarkan jarak target dan kondisi energi bot.

- **Energy Management**

Saat energi rendah, bot mengurangi firepower agar peluang bertahan hidup lebih tinggi.

4.2.3 Prosedur

Strategi Adaptive Greedy Combat memiliki kompleksitas waktu $O(1)$, karena setiap tick hanya melakukan sejumlah operasi tetap berupa perhitungan matematika, percabangan, dan pembaruan variabel. Tidak terdapat proses pencarian state ataupun eksplorasi seluruh arena.

Dengan kompleksitas konstan tersebut, bot dapat mengambil keputusan dengan cepat pada lingkungan Robocode yang bersifat real-time.

4.3 Pengujian

Pengujian dilakukan dalam format 1 vs 1 antara bot utama dengan bot asisten. Berikut adalah hasil pengujian:

4.3.1 Pengujian 1: [Bot Utama vs Bot Alternatif 2]

Menggunakan arena berukuran 800 x 600. Turn Timeout 30000. Max Turn Count 5000.

Parameter	Detail
Bot Utama	UjangKeduDorDor
Bot Asisten	UjangKeduPiwPiw
Jumlah Ronde	3
Hasil	Menang, 347
Kondisi Unik	-

Results for 3 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	UjangKeduDorDor 1.8	347	100	20	186	26	14	0	2	1	0
2	UjangKeduPiwPiw 1.3	310	50	10	216	21	12	0	1	2	0

Pada pengujian pertama, bot utama UjangKeduDorDor diadu melawan UjangKeduPiwPiw dalam arena berukuran 800 × 600 selama 3 ronde. Selama

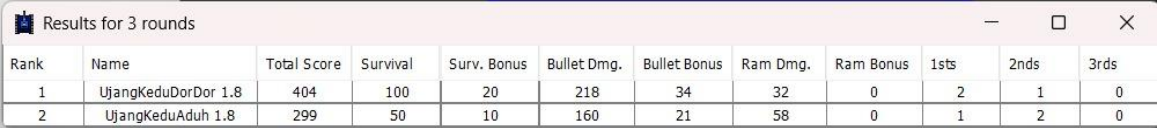
pertandingan berlangsung, UjangKeduPiwPiw menggunakan pola gerakan zigzag dan pemilihan firepower berdasarkan jarak musuh. Sementara itu, UjangKeduDorDor memanfaatkan movement adaptif, predictive targeting, serta distance control.

Hasil pengujian menunjukkan UjangKeduDorDor berhasil memenangkan pertandingan dengan skor 347. Keunggulan bot utama terlihat pada kemampuan dodge dan predictive aiming yang membuat serangan lebih efektif dibandingkan strategi lawan yang hanya menggunakan targeting sederhana berdasarkan posisi musuh saat ini.

4.3.2 Pengujian 2: [Bot Utama vs Bot Alternatif 3]

Menggunakan arena berukuran 800 x 600. Turn Timeout 30000. Max Turn Count 5000.

Parameter	Detail
Bot Utama	UjangKeduDorDor
Bot Asisten	UjangKeduAduh
Jumlah Ronde	3
Hasil	Menang, 404
Kondisi Unik	UjangKeduAduh nyangkut ditembok dan tidak bergerak



Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	UjangKeduDorDor 1.8	404	100	20	218	34	32	0	2	1	0
2	UjangKeduAduh 1.8	299	50	10	160	21	58	0	1	2	0

Pada pengujian kedua, bot utama UjangKeduDorDor diadu melawan UjangKeduAduh dalam arena yang sama dengan jumlah ronde sebanyak 3. Strategi UjangKeduAduh berfokus pada pengejaran musuh secara agresif dan penggunaan firepower maksimum saat target terdeteksi.

Selama pertandingan ditemukan kondisi unik yaitu UjangKeduAduh beberapa kali mengalami kondisi terjebak di dekat dinding arena sehingga pergerakannya menjadi tidak efektif. Kondisi tersebut menyebabkan bot kehilangan momentum dan menjadi target yang lebih mudah diserang.

Hasil akhir menunjukkan UjangKeduDorDor memenangkan pertandingan dengan skor 404. Sistem anti-wall movement dan movement adaptif pada bot utama terbukti lebih efektif dibanding strategi agresif yang digunakan lawan.

4.3.3 Pengujian 3: [Bot Utama vs Bot Alternatif 4]

Menggunakan arena berukuran 800 x 600. Turn Timeout 30000. Max Turn Count 5000.

Parameter	Detail
Bot Utama	UjangKeduDorDor
Bot Asisten	UjangKeduAwoo
Jumlah Ronde	3
Hasil	Menang, 404
Kondisi Unik	UjangKeduDorDor hampir kalah di semua ronde, namun bisa mengembalikan keadaan

Results for 3 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	UjangKeduDorDor 1.8	404	100	20	208	19	56	0	2	1	0
2	UjangKeduAwoo 1.5	351	50	10	220	19	52	0	1	2	0

Pada pengujian ketiga, bot utama UjangKeduDorDor diadu melawan UjangKeduAwoo dalam arena berukuran 800×600 selama 3 ronde. UjangKeduAwoo menggunakan strategi yang lebih kompleks dengan predictive targeting, orbit movement, distance control, dan anti-targeting movement.

Pertandingan berlangsung lebih kompetitif dibanding pengujian sebelumnya. Pada beberapa ronde, UjangKeduDorDor sempat berada pada kondisi energi yang lebih rendah dan hampir mengalami kekalahan. Namun, penggunaan adaptive movement, dodge system, dan pengaturan jarak tempur memungkinkan bot utama untuk secara bertahap membalikkan keadaan.

Hasil akhir menunjukkan UjangKeduDorDor tetap berhasil memenangkan pertandingan dengan skor 404. Pengujian ini menunjukkan bahwa strategi Adaptive Greedy Combat memiliki kemampuan adaptasi yang cukup baik terhadap lawan dengan strategi yang lebih kompleks.

4.4 Analisis Hasil Pengujian

Strategi Adaptive Greedy Combat pada bot UjangKeduDorDor secara umum berhasil menunjukkan performa yang baik dan konsisten. Bot utama berhasil memenangkan seluruh pertandingan melawan ketiga bot alternatif yang digunakan sebagai bot asisten. Hasil ini menunjukkan bahwa strategi greedy yang diterapkan mampu memilih aksi lokal terbaik secara efektif pada setiap tick permainan.

Pada pengujian pertama melawan UjangKeduPiwPiw, strategi greedy berhasil memperoleh hasil yang optimal karena bot lawan menggunakan pendekatan yang lebih sederhana, yaitu movement zigzag dan pemilihan firepower berdasarkan jarak. Strategi

Adaptive Greedy Combat memiliki keunggulan berupa predictive targeting, distance control, dan dodge movement sehingga bot mampu menghasilkan serangan yang lebih akurat sekaligus mempertahankan posisi yang lebih aman.

Pada pengujian kedua melawan UjangKeduAduh, hasil yang diperoleh juga sangat baik. Bot lawan menggunakan strategi agresif dengan fokus mengejar target dan menembakkan firepower maksimum. Namun ditemukan kondisi unik berupa bot lawan yang beberapa kali terjebak di dekat tembok arena. Kondisi tersebut menyebabkan strategi greedy lawan gagal mengambil keputusan yang optimal karena posisi yang buruk mengurangi efektivitas pergerakan dan meningkatkan risiko terkena serangan. Sebaliknya, UjangKeduDorDor memiliki sistem anti-wall movement sehingga dapat menghindari kondisi tersebut.

Pengujian ketiga melawan UjangKeduAwoo menghasilkan pertandingan yang paling kompetitif. Bot lawan menggunakan strategi yang lebih kompleks seperti predictive targeting, orbit movement, dan dynamic distance control. Pada beberapa ronde UjangKeduDorDor hampir mengalami kekalahan karena lawan mampu memberikan tekanan yang cukup besar. Meskipun demikian, bot utama masih dapat membalikkan keadaan melalui mekanisme adaptive movement, dodge system, dan pengaturan jarak tempur. Hal ini menunjukkan bahwa strategi greedy yang dipilih memiliki kemampuan adaptasi yang cukup baik terhadap perubahan kondisi pertandingan.

Meskipun secara keseluruhan strategi greedy berhasil memperoleh hasil yang baik, strategi ini tidak selalu menghasilkan solusi optimal pada semua kondisi. Pendekatan greedy hanya mempertimbangkan kondisi lokal saat ini tanpa memperhitungkan konsekuensi jangka panjang. Sebagai contoh, mekanisme deteksi peluru pada UjangKeduDorDor menggunakan penurunan energi musuh sebagai indikator tembakan. Pendekatan tersebut tidak selalu akurat karena penurunan energi dapat dipengaruhi faktor lain. Selain itu, predictive targeting yang digunakan masih menggunakan prediksi linear sederhana sehingga performanya dapat menurun ketika menghadapi musuh dengan pola gerakan yang sangat acak.

Jika dibandingkan antar pengujian, performa terbaik terlihat pada pengujian kedua karena bot lawan mengalami kondisi yang menguntungkan bagi strategi utama. Tingkat kesulitan sedang terlihat pada pengujian pertama karena lawan memiliki pola movement yang

cukup aktif namun belum memiliki mekanisme prediksi. Pengujian ketiga menjadi pengujian paling sulit karena lawan menggunakan strategi yang lebih kompleks dan adaptif. Dari hasil tersebut dapat disimpulkan bahwa strategi Adaptive Greedy Combat mampu memberikan keseimbangan yang baik antara agresivitas, kemampuan bertahan hidup, dan efektivitas pengambilan keputusan pada lingkungan Robocode yang bersifat real-time.

BAB 5 KESIMPULAN DAN SARAN

5.1 Kesimpulan

Pada tugas ini telah dilakukan implementasi algoritma greedy pada permainan Robocode Tank Royale melalui pengembangan bot dengan berbagai strategi pengambilan keputusan. Sebanyak empat alternatif solusi greedy berhasil dirancang dan dianalisis, yaitu Adaptive Greedy Combat, Distance-Based Combat, Aggressive Pursuit, dan Predictive Dynamic Combat. Masing-masing strategi menggunakan heuristic yang berbeda dalam menentukan aksi terbaik pada setiap tick permainan.

Dari keempat alternatif yang telah dieksplorasi, strategi Adaptive Greedy Combat dipilih untuk diimplementasikan sebagai strategi utama karena memiliki keseimbangan yang baik antara kemampuan menyerang, bertahan, dan adaptasi terhadap kondisi arena. Strategi ini menggabungkan beberapa mekanisme seperti predictive targeting, distance control, movement adaptation, anti-wall behavior, energy management, dan bullet dodge untuk menentukan keputusan lokal terbaik pada setiap kondisi permainan.

Berdasarkan hasil pengujian yang telah dilakukan terhadap tiga bot alternatif, bot utama UjangKeduDorDor berhasil memenangkan seluruh pertandingan. Hasil tersebut menunjukkan bahwa strategi greedy yang diterapkan mampu bekerja secara efektif pada lingkungan Robocode yang bersifat real-time. Strategi ini mampu menyesuaikan perilaku berdasarkan kondisi arena dan karakteristik lawan sehingga dapat mempertahankan performa yang stabil.

Melalui tugas ini dapat dipahami bahwa algoritma greedy cukup efektif digunakan pada lingkungan yang membutuhkan pengambilan keputusan cepat. Namun, strategi greedy tidak selalu menghasilkan solusi global terbaik karena keputusan diambil berdasarkan kondisi lokal saat ini tanpa mempertimbangkan seluruh kemungkinan kondisi di masa depan. Oleh karena itu, perancangan heuristic menjadi faktor yang sangat penting dalam menentukan performa algoritma.

5.2 Saran

Untuk pengembangan lebih lanjut, terdapat beberapa hal yang dapat dilakukan untuk meningkatkan performa bot yang telah dibuat. Salah satu pengembangan yang dapat dilakukan adalah memperbaiki sistem predictive targeting agar tidak hanya menggunakan

prediksi gerakan linear sederhana, tetapi juga mempertimbangkan perubahan arah musuh secara dinamis.

Selain itu, sistem movement dapat dikembangkan dengan pola dodge yang lebih adaptif agar bot lebih sulit diprediksi lawan. Penggunaan teknik movement yang lebih kompleks seperti wave surfing atau movement berbasis pola serangan musuh juga dapat dipertimbangkan.

Pengujian yang dilakukan pada tugas ini masih menggunakan skenario pertandingan satu lawan satu dengan jumlah ronde terbatas. Pada pengembangan selanjutnya, pengujian dapat dilakukan dengan jumlah ronde yang lebih banyak atau menggunakan variasi ukuran arena dan karakteristik lawan yang berbeda untuk memperoleh hasil evaluasi yang lebih representatif.

Selain algoritma greedy, pendekatan lain seperti algoritma genetika, fuzzy logic, reinforcement learning, atau kombinasi beberapa algoritma (hybrid approach) juga dapat dicoba untuk meningkatkan kemampuan pengambilan keputusan bot secara lebih adaptif dan optimal.

LAMPIRAN

Lampiran A: Tautan Repository GitHub

Tautan Repository GitHub: https://github.com/dikihurrisyail/Tubes_UjangKedu.git

Lampiran B: Tautan Video (jika ada)

Tautan Video: <https://youtu.be/fhZ0DcV9muM>

DAFTAR PUSTAKA

- [1] [Nama Penulis]. (Tahun). Judul Buku/Artikel. Penerbit/Jurnal.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- [3] Robocode Tank Royale. (2024). Official Documentation. Diakses dari <https://robocode-dev.github.io/tank-royale/>